E-7 Operations Engine: Comprehensive Implementation Plan

1. Executive Summary

The Operations Engine (E-7) serves as the core orchestration and execution layer of the AI ecosystem. By consolidating findings from six separate research reports, this document outlines a unified architecture that integrates **Task Intelligence**, **Workflow Automation**, and **Scheduling & Resource Optimization** into a single cohesive platform.

The core philosophy of this engine is the strict separation of AI reasoning from deterministic execution. Generative AI (LLMs) is restricted to planning and task intelligence, while mathematically rigorous constraint solvers and rule-based workflow engines handle scheduling and business processes. This ensures reliability, scalability, and compliance with the **Open-Source AI First** strategy.

A primary design focus is **integration flexibility**. The engine is not just a standalone application; it is built API-first so that its intelligence can be easily plugged into existing third-party software solutions.

> [!NOTE] ***Developer Flexibility:*** While this document outlines a strong strategic direction, the development team is not rigidly bound to these exact tools or models. Developers are encouraged to remain flexible and suggest architectural changes, alternative models, or different tech stacks if they identify more efficient, cost-effective, or performant solutions during implementation.

2. Core Operational Modes

2.1 Task Intelligence Mode

Converts natural language institutional goals into structured, executable Work Breakdown Structures (WBS) and monitors project health.

- **Capabilities**: Goal-to-task decomposition, dependency analysis, effort estimation, resource recommendation, milestone planning, and risk/blocker detection.
- **LLM Reasoning Layer (Low-Cost & Open-Source Focus)**:
  - **Primary Target - Sarvam AI**: We will first evaluate Sarvam AI to determine if it meets the reasoning and accuracy requirements for goal

decomposition and effort estimation. If successful, this will be the primary model.
    - **Fallback/Open-Source Models**: If Sarvam AI does not meet the requirements, we will fall back to low-cost, high-accuracy open-source models such as **Llama 3.x**, Qwen 3, Mistral, or DeepSeek-R1.
    - **Cost & Accuracy Mandate**: The absolute priority for the reasoning layer is maintaining extremely low inference costs while delivering high accuracy. Expensive commercial APIs should be avoided in production.
- **Orchestration & Algorithms**: LangGraph / LangChain for multi-agent planning. NetworkX and Critical Path Method (CPM) for deterministic dependency graph generation. XGBoost / LightGBM for historical effort estimation.

2.2 Workflow Mode

Automates institutional and administrative processes through no-code BPMN-based workflow orchestration.

- **Capabilities**: Visual workflow modeling, conditional routing, parallel/sequential execution, SLA monitoring, escalation handling, and comprehensive audit trails.
- **Execution Engines**:
    - **Flowable**: Selected for no-code BPMN 2.0 visual design and general-purpose approval chains.
    - **Temporal**: Complements Flowable by providing durable execution for code-first, long-running background steps.
- **Business Rules**: Drools or Easy Rules for deterministic state transition evaluation.

2.3 Scheduling & Resource Optimization Mode

Generates highly optimized, conflict-free schedules using exact mathematical constraint solvers. **(Explicitly Non-LLM)**

- **Capabilities**: Academic timetabling, examination scheduling, faculty/room allocation, incremental rescheduling during disruptions, and CalDAV/iCal synchronization.
- **Constraint Solvers**:
    - **Google OR-Tools (CP-SAT)**: Primary general-purpose constraint programming and optimization solver.

- **FET (Free Educational Timetabling)**: Purpose-built for rapid generation of complex school and exam timetables (AGPL licensed).
- **Timefold Solver**: Java/Kotlin-based solver for advanced room, invigilator, and employee rostering.

3. Recommended Architecture & Tech Stack

Following the Open-Source AI First approach, the E-7 engine operates as a set of decoupled microservices:

Layer	Recommended Technology	Purpose
Backend Framework	FastAPI (Python), Spring Boot (Java)	REST and Event-driven microservices for external API integration
Workflow Engine	Flowable, Temporal (Apache 2.0 / MIT)	Process automation, durable execution
Scheduling Solver	OR-Tools, FET, Timefold	Constraint optimization, timetabling
AI Orchestration	LangGraph, LangChain	Multi-agent task planning
Primary Database	PostgreSQL	Transactional storage (tasks, workflows)
Vector Database	Qdrant / pgvector	Semantic task search
Messaging / Events	Apache Kafka, RabbitMQ	Event streaming and notifications

4. Integration & Deployment Strategy

Deployment Modes

The engine is designed API-first to support two distinct modes of operation:

1. **Universal Mode (Standalone Site)**: A dedicated platform/site where users can directly interact with the AI. In this mode, the engine provides general-purpose planning, workflow, and scheduling intelligence based on general data.
2. **Embedded/Integrated Mode (API-Driven)**: The engine exposes robust APIs designed to connect to existing third-party software solutions. When integrated, the engine's interface and intelligence operate entirely within the external system, providing responses and scheduling optimized specifically against the client's proprietary dataset (rather than general responses).

Phased Implementation Roadmap

- **Phase 1**: Stand up API-first backend infrastructure (FastAPI) and evaluate **Sarvam AI** for task intelligence. Fallback to Llama 3 if needed.
- **Phase 2**: Integrate Google OR-Tools/NetworkX for deterministic dependency tracking and basic scheduling via API.
- **Phase 3**: Integrate Flowable to model and execute foundational approval workflows.
- **Phase 4**: Pilot FET for automated educational timetabling on a subset of school data.
- **Phase 5**: Enable continuous real-time monitoring and incremental rescheduling for disruptions.